

SD	Sistemas Distribuidos
	Prácticas
#1b	Anexo a la tecnología de Sockets (con Java)

Ejercicio Práctico Puente: De la Calculadora al Sistema de Riego

Una vez realizada la práctica de contacto con sockets mediante el ejemplo clásico, vamos a modificar el contenido manteniendo la estructura concurrente, lo que nos servirá para implementar directamente la base de la siguiente práctica de *Water Management*.

Objetivo Evolucionar el servidor concurrente básico para construir las piezas iniciales utilizando los nombres de clases obligatorios de la práctica final (WM_Central y WM_WS_M), asegurando que este código sea 100% reutilizable en vuestro repositorio definitivo.

Componentes del Sistema

1. WM_Central (**Servidor**): Aplicación centralizada que hereda la lógica del servidor concurrente con hilos (Threads), quedando a la escucha en un puerto para aceptar las conexiones simultáneas de múltiples estaciones.
2. WM_WS_M (**Cliente / Monitor**): El módulo monitor de la estación de riego que utiliza la estructura del cliente básico para conectarse a la central e identificarse.

Protocolo de Mensajería Inicial

Aprovechando la base de envío de texto de la práctica anterior, adaptamos el mensaje a una trama estructurada por campos separados por el carácter almohadilla (#):

REGISTRO#<ID_ESTACION>#<UBICACION>

Ejemplo de trama enviada por el cliente (WM_WS_M):

REGISTRO#WS-04#River Park

Requisitos de Funcionamiento

- El servidor (WM_Central) recibe la trama, parsea los campos utilizando el separador #, valida los datos y muestra por consola la información de la estación conectada.
- Como respuesta, el servidor devuelve al cliente una confirmación estructurada utilizando los flujos de datos:
STATUS#OK#Estacion registrada correctamente
- El cliente (WM_WS_M) lee la respuesta del flujo de entrada, la imprime por pantalla y realiza un cierre limpio de la conexión (close()).
- El servidor mantiene la concurrencia intacta para atender a múltiples estaciones simultáneamente sin bloqueos.